UNIVERSIDAD NACIONAL JOSÉ MARÍA ARGUEDAS

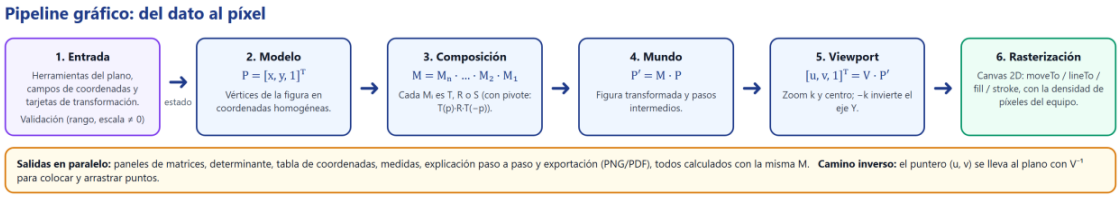
Facultad de Ingeniería · Escuela Profesional de Ingeniería de Sistemas

Computación Gráfica (IIAD65) · Semestre 2026-II

MANUAL TÉCNICO

GeoMatriz 2D

Arquitectura, algoritmos, ejecución y pruebas



Grupo 3 · Oscco Zarabia Rider Gilmar · Vargas Alhuay Hernan · Chilingano Lazo Albert

Docente: Mtro. Jhamer Anderson Rojas Silva

Andahuaylas, Perú · octubre de 2026

Contenido

1. Tecnologías y versiones 3

2. Arquitectura 3

3. Pipeline gráfico 4

4. Descripción de los módulos 5

5. Fragmentos de código clave 5

6. Estructura del repositorio y ejecución 7

7. Pruebas y rendimiento 8

8. Limitaciones conocidas y mejoras posibles 8

1. Tecnologías y versiones

Tabla 1

Tecnologías y versiones utilizadas

Tecnología / herramienta	Versión	Uso
HTML5	Estándar vivo (WHATWG)	Estructura de la interfaz y ventanas <dialog>
CSS3	Grid, Flexbox, variables	Diseño adaptable y tema de colores
JavaScript	ES2015 o posterior	Toda la lógica, sin bibliotecas
API Canvas 2D	Integrada en el navegador	Rasterización del plano y exportación PNG
Navegadores de prueba	Chrome 155, Edge 155	Ejecución y pruebas
Node.js	24.15	Solo para verificar la sintaxis (node --check); no es necesario para usar el programa
Git	2.54	Control de versiones
Sistema operativo de desarrollo	Windows 11	—

Fuente: Elaboración propia.

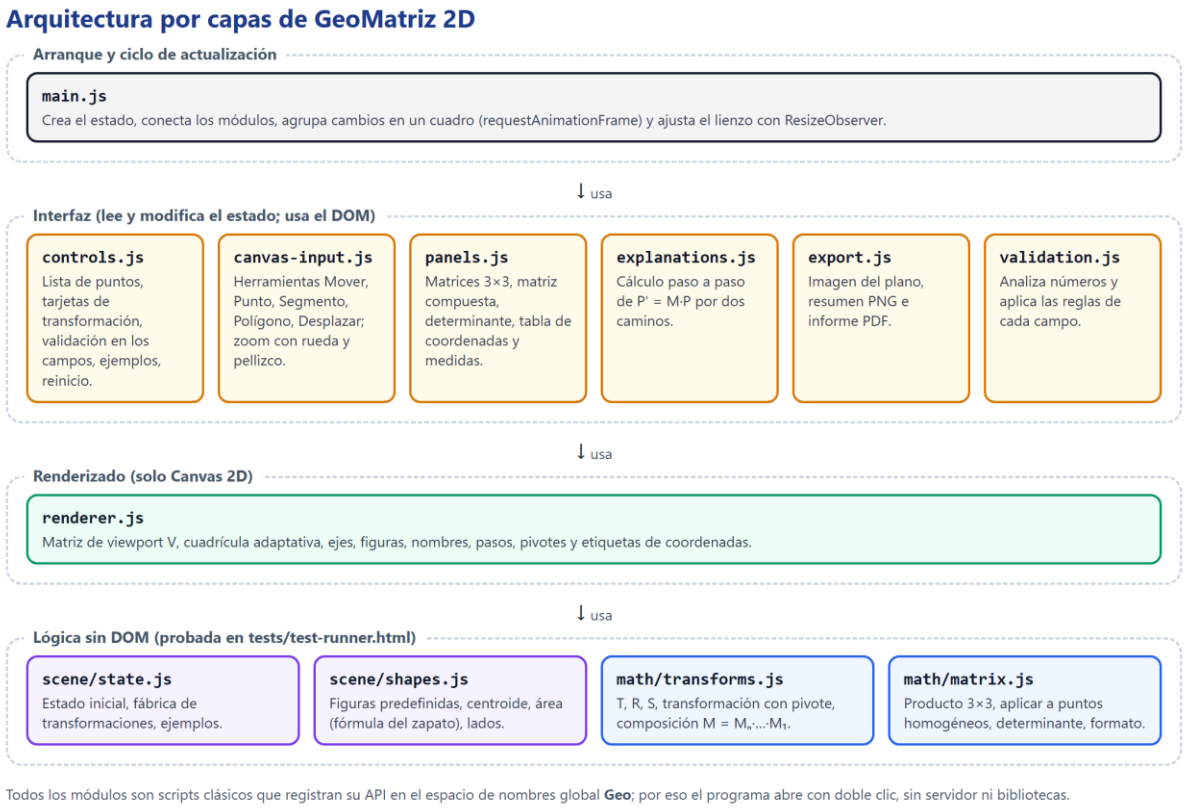
Bibliotecas de terceros: ninguna. El producto de matrices, las transformaciones, la composición, el pivote, el viewport, la cuadrícula y la exportación fueron programados por el grupo, como exige la guía.

2. Arquitectura

El programa está dividido en capas. Cada capa solo usa las de abajo: la lógica (math y scene) no conoce el DOM y puede probarse sola; el renderizador solo dibuja en el lienzo; la interfaz conecta los controles con el estado; y main.js arranca todo y agrupa los cambios en un único redibujado por cuadro.

Figura 1

Diagrama de módulos y capas



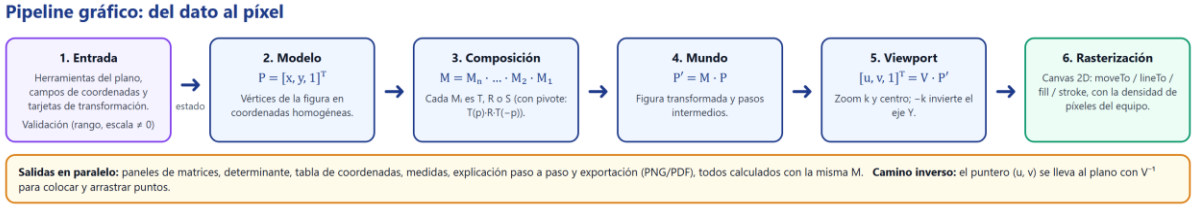
Fuente: Elaboración propia.

Todos los archivos son scripts clásicos (no módulos ES) que registran su API en el espacio de nombres global `window.Geo`. Esta decisión permite abrir el programa con doble clic desde el sistema de archivos (`file://`), sin servidor, lo que garantiza la demostración en el laboratorio. El orden de carga en `index.html` respeta las dependencias: math → scene → validation → render → ui → main.

3. Pipeline gráfico

Figura 2

Pipeline gráfico: de la entrada del usuario al píxel



Fuente: Elaboración propia.

- **Entrada:** herramientas del plano, campos de coordenadas y tarjetas de transformación; cada valor pasa por la validación.
- **Modelo:** vértices en coordenadas homogéneas $P = [x, y, 1]^T$.
- **Composición:** $M = M_n \cdot \ldots \cdot M_1$, con T, R, S y la variante con pivote.
- **Mundo:** $P' = M \cdot P$ para cada vértice y para cada paso intermedio.
- **Viewport:** $[u, v, 1]^T = V \cdot P'$ con zoom k y centro de la vista; −k invierte el eje Y.
- **Rasterización:** Canvas 2D (moveTo, lineTo, fill, stroke) escalado a la densidad de píxeles del equipo.

4. Descripción de los módulos

Tabla 2

Módulos, algoritmos y justificación

Módulo	Qué implementa	Fórmulas / algoritmo	Por qué así
`math/matrix.js`	Matrices 3×3	$C[i][j] = \Sigma_k A[i][k] \cdot B[k][j]$; det por cofactores	Arreglo de filas: coincide con la notación del pizarrón
`math/transforms.js`	T, R, S, pivote, composición	$T(p) \cdot M \cdot T(-p)$; $M = M_n \cdot \ldots \cdot M_1$	Una sola operación (producto) para todo
`scene/shapes.js`	Figuras y medidas	Centroide ($\Sigma x/n$, $\Sigma y/n$); área por fórmula del zapato; tipo según n.º de puntos	Medidas exactas para comprobar $\lvert\det(M)\rvert \cdot A$
`scene/state.js`	Estado y ejemplos	Escena vacía inicial; fábrica de transformaciones	«Reiniciar» vuelve a un estado conocido
`render/renderer.js`	Dibujo en Canvas	$V = [[k,0,c_u],[0,-k,c_v],[0,0,1]]$; paso de cuadrícula $1\text{-}2\text{-}5 \cdot 10^n$	El viewport también es una matriz homogénea
`ui/canvas-input.js`	Herramientas del plano	V^{-1} del puntero; zoom anclado $c_x' = a_x - (u - W/2)/k'$	El punto bajo el cursor no se mueve al hacer zoom
`ui/validation.js`	Validación	Rangos por campo; escala ≠ 0; coma decimal	Nada se corrige en silencio
`ui/controls.js`	Controles	Lista de puntos, tarjetas, sentido de giro = signo de θ	Valores siempre válidos en el estado
`ui/panels.js`, `explanations.js`	Resultados	Matrices visibles; $P' = M \cdot P$ desarrollado	Dos caminos que deben coincidir
`ui/export.js`	Exportación	Redibujo a 3200×2000 px; informe HTML imprimible	Sin bibliotecas de PDF

Fuente: Elaboración propia.

5. Fragmentos de código clave

Los fragmentos se copian tal cual del código fuente. Los números de ecuación y de algoritmo que aparecen en los comentarios (por ejemplo, «Ecuación (9)» o «algoritmo A4») remiten al documento de diseño del repositorio, `docs/diseno.md`.

Código 1. Aplicar M a un punto homogéneo (math/matrix.js)

```
/**
 * Aplica M a un punto: P' = M · [x, y, 1]ᵀ (algoritmo A4).
 * La última fila de nuestras matrices es siempre [0 0 1], así que w' = 1.
 */
function applyToPoint(M, point) {
  return {
    x: M[0][0] * point.x + M[0][1] * point.y + M[0][2],
    y: M[1][0] * point.x + M[1][1] * point.y + M[1][2]
  };
}
```

Como la última fila de las matrices afines es siempre [0 0 1], basta calcular x′ e y′; w′ = 1.

Código 2. Matriz de rotación R(θ) (math/transforms.js)

```
/** Ecuación (3): matriz de rotación R(θ) respecto al origen. */
function rotation(degrees) {
  const t = toRadians(degrees);
  const c = Math.cos(t);
  const s = Math.sin(t);
  return [
    [c, -s, 0],
    [s,  c, 0],
    [0,  0, 1]
  ];
}
```

Código 3. Matriz de una transformación, con pivote si corresponde

```
/** Construye la matriz 3×3 de una transformación de la escena (algoritmo A2). */
function matrixFor(transform, centroid) {
  const p = transform.params;
  if (transform.type === 'T') {
    return translation(p.tx, p.ty);
  }
  const pivot = resolvePivot(p.pivot, centroid || { x: 0, y: 0 });
  const base = transform.type === 'R' ? rotation(p.angle) : scaling(p.sx, p.sy);
  if (pivot.x === 0 && pivot.y === 0) return base;
  return aroundPivot(base, pivot.x, pivot.y);
}
```

Código 4. Zoom que mantiene fijo el punto bajo el cursor (ui/canvas-input.js)

```
/**
 * Cambia el zoom manteniendo fijo el punto del plano bajo el píxel (u, v).
 * Ecuación (9):  cx = x − (u − W/2) / k     cy = y + (v − H/2) / k
 */
function setZoom(z, u, v) {
  const size = renderer.size();
  if (u === undefined) { u = size.width / 2; v = size.height / 2; }
  const anchor = renderer.toWorld(u, v, state.view);
  const k = Math.min(ZOOM_MAX, Math.max(ZOOM_MIN, z));
  state.view.zoom = k;
  state.view.center = {
    x: anchor.x - (u - size.width / 2) / k,
    y: anchor.y + (v - size.height / 2) / k
  };
  state.view.manualZoom = true;
```

```
  o.requestRender();
}
```

Código 5. Validación de un campo (ui/validation.js)

```
/**
 * Valida un texto según una regla.
 * @param {string} text
 * @param {string} ruleName  clave de RULES
 */
function validate(text, ruleName) {
  const rule = RULES[ruleName];
  const value = parseNumber(text);
  if (Number.isNaN(value)) {
    return { ok: false, error: 'Ingrese un número válido (por ejemplo 2 o 1,5).' };
  }
  if (value < rule.min || value > rule.max) {
    return { ok: false, error: rule.message };
  }
  if (rule.nonZero && Math.abs(value) < rule.nonZero) {
    return {
      ok: false,
      error: 'La escala no puede ser 0 (mínimo ±' + rule.nonZero + '): la figura
colapsaría y det(M) = 0.'
    };
  }
  return { ok: true, value: value };
}
```

6. Estructura del repositorio y ejecución

Estructura de carpetas

```
COMP_GRAFICA PROYECT/
├── README.md                 descripción y cómo ejecutar
├── src/                      código fuente
│   ├── index.html            punto de entrada (doble clic)
│   ├── css/estilos.css
│   └── js/
│       ├── math/             matrix.js, transforms.js         (sin DOM)
│       ├── scene/            shapes.js, state.js              (sin DOM)
│       ├── render/           renderer.js                      (Canvas 2D)
│       ├── ui/               controls.js, canvas-input.js, panels.js,
│       │                     explanations.js, validation.js, export.js
│       └── main.js           arranque y ciclo de actualización
├── tests/                    test-runner.html, math.test.js
├── assets/                   capturas/ y diagramas/
└── docs/                     planificación, diseño, pruebas, entrega/
```

Compilar: no hace falta. El código se ejecuta tal cual en el navegador.

Ejecutar: abrir `src/index.html` con doble clic en Chrome, Edge o Firefox.

Probar: abrir `tests/test-runner.html`; las pruebas se ejecutan al cargar y el botón «Medir rendimiento» mide los tiempos.

Modificar: editar los archivos de `src/js` y recargar la página (Ctrl + F5). Para comprobar la sintaxis puede usarse `node --check archivo.js`. Si se agrega un archivo, debe incluirse en `index.html` respetando el orden de carga.

7. Pruebas y rendimiento

La página de pruebas carga los mismos archivos de lógica que el programa y compara cada resultado con el valor calculado a mano, con tolerancia $1 \cdot 10^{-9}$. Resultado: **39 de 39 pruebas aprobadas.**

Tabla 3

Grupos de pruebas automáticas

Pruebas	Qué verifican
CP01–CP22	Núcleo matemático: identidad, T, R, S, pivote, composición, orden, determinante, viewport, pasos
CP23–CP27	Apoyo: centroide fijo, formato numérico, lados, área |det(M)|·A, tipo de figura
CV01–CV12	Validación: vacío, texto, escala 0, coma decimal, rangos, Infinity, signo menos, reflexión

Fuente: Elaboración propia.

Tabla 4

Rendimiento: 100 dibujos completos con 10 transformaciones, pasos y comparación activados

ID	Vértices	Tiempo medio (ms)	Cuadros/s equivalentes
CR01	3	0.442	2262
CR02	12	0.489	2045
CR03	100	2.412	415

Fuente: Medición propia en Windows 11, navegador basado en Chromium (Chrome 155 / Edge 155).

Además se registraron 79 pruebas manuales de interfaz (anchos de 375, 768, 1024, 1440 y 1600 px, herramientas, validación, exportación) en `docs/pruebas.md`, junto con los errores encontrados y su corrección.

8. Limitaciones conocidas y mejoras posibles

Tabla 5

Limitaciones y mejoras

Limitación	Mejora posible
Una sola figura a la vez	Varias figuras con su propia lista de transformaciones
No guarda ni abre escenas	Exportar e importar la escena en formato JSON

Limitación	Mejora posible
Sin recorte de polígonos	Algoritmo de Sutherland–Hodgman contra una ventana rectangular
El PDF depende del diálogo de impresión del navegador	Generar el PDF directamente con código propio
Probado en navegadores basados en Chromium	Ampliar las pruebas a Firefox y Safari y al navegador del Lab. 05
Sin animación de la transformación	Interpolar la figura entre P y P′ para ver el movimiento

Fuente: Elaboración propia.